Module 7: Tensor Parallelism

Splitting Matrix Multiplication Inside Each Layer

BRAIN team

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

Lecture map

1. Start from one matrix multiplication
2. Inside a Transformer block
3. Cost and placement

Guiding question

How can several GPUs compute one Transformer layer without reconstructing every intermediate tensor?

What should be clear by the end?

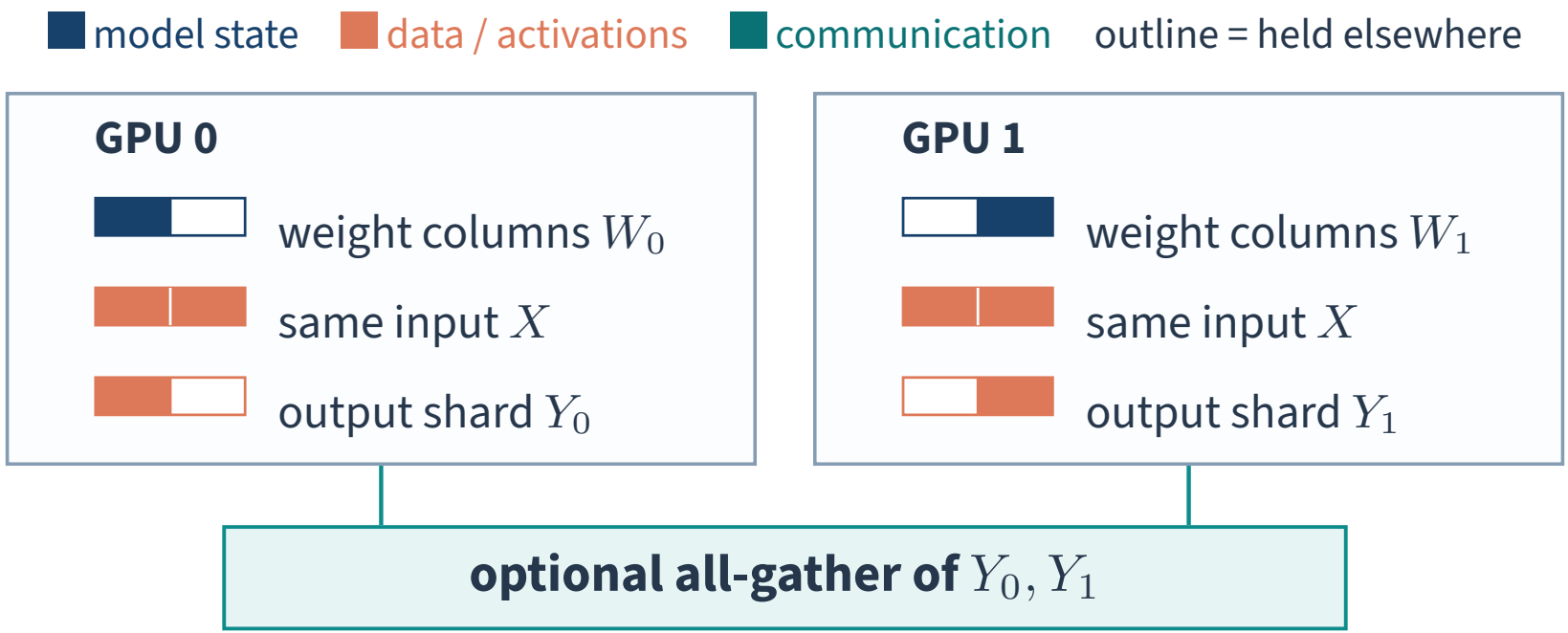
1. Derive column- and row-parallel linear layers from matrix multiplication.
2. Track replicated and sharded tensor layouts through an MLP and attention block.
3. Connect forward collectives with their backward counterparts.
4. Choose tensor-parallel degree from layer shapes and topology.

PART 1

Start from one matrix multiplication

1. Start from one matrix multiplication
2. Inside a Transformer block
3. Cost and placement

Parallelism map: TP splits a layer's width



What changes? Devices cooperate on the *same* tokens and hold different weight slices. Here $Y = [XW_0 \; XW_1]$.

A following row-parallel layer can consume sharded outputs without an intermediate gather.

Column parallelism shards output features

Let $Y = XW$ and split W by columns:

$$W = [W_1\ W_2\ \cdots\ W_p]$$

$$Y = [XW_1\ XW_2\ \cdots\ XW_p]$$

Local computation

Every rank receives replicated X, stores one W_i, and produces one output-feature shard Y_i without forward communication.

Layout change

Input is replicated. Output is sharded across the hidden-feature axis.

Row parallelism shards input features

Split both input and weight along their contracting dimension:

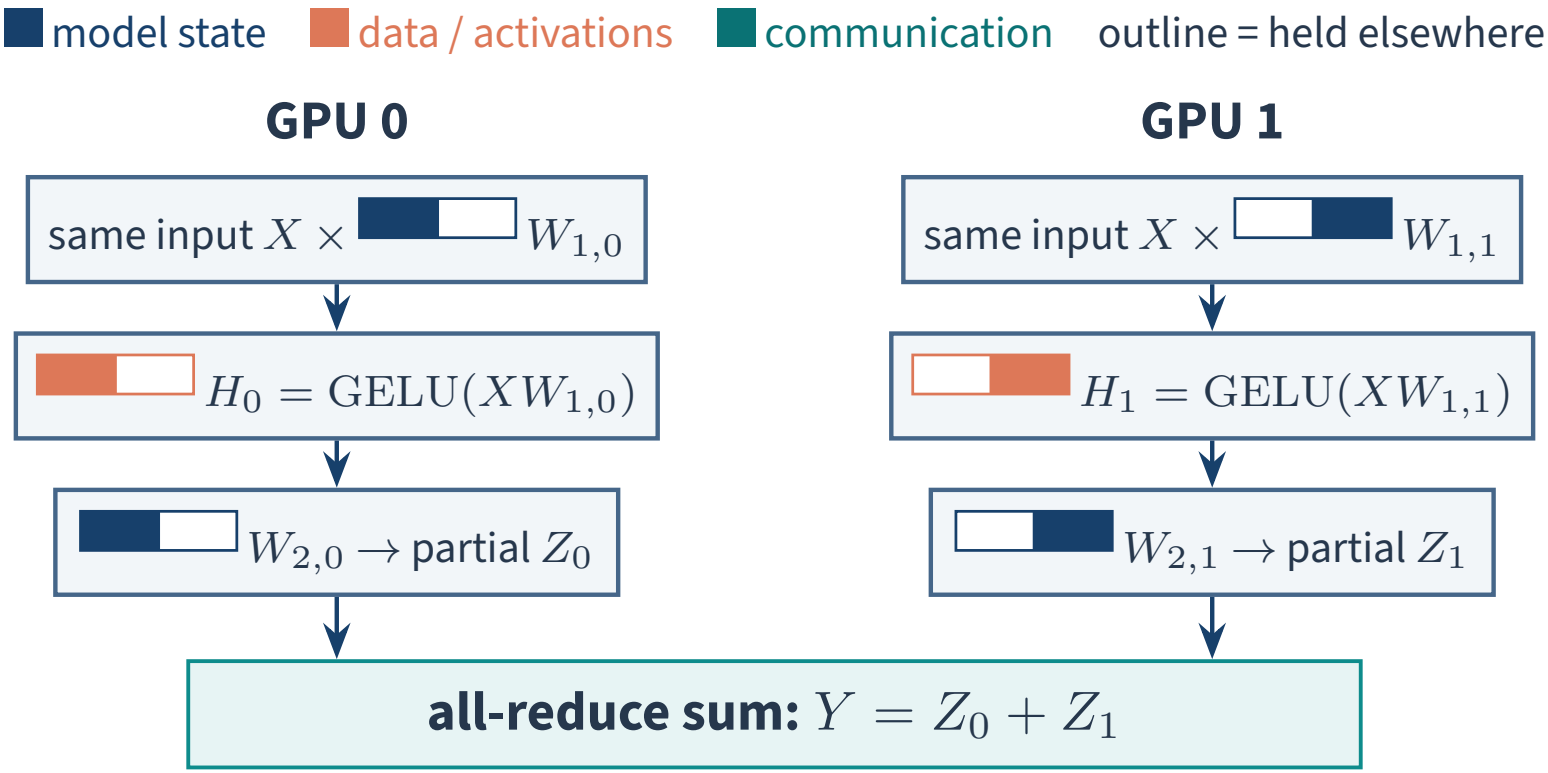
$$X = [X_1\ X_2\ \cdots\ X_p], \qquad W = \begin{bmatrix} W_1 \\ W_2 \\ \vdots \\ W_p \end{bmatrix}$$

$$Y = \sum_{i=1}^{p} X_i W_i$$

Local computation and collective

Each rank computes a partial replicated-shape output. An all-reduce sum produces the full Y on every rank.

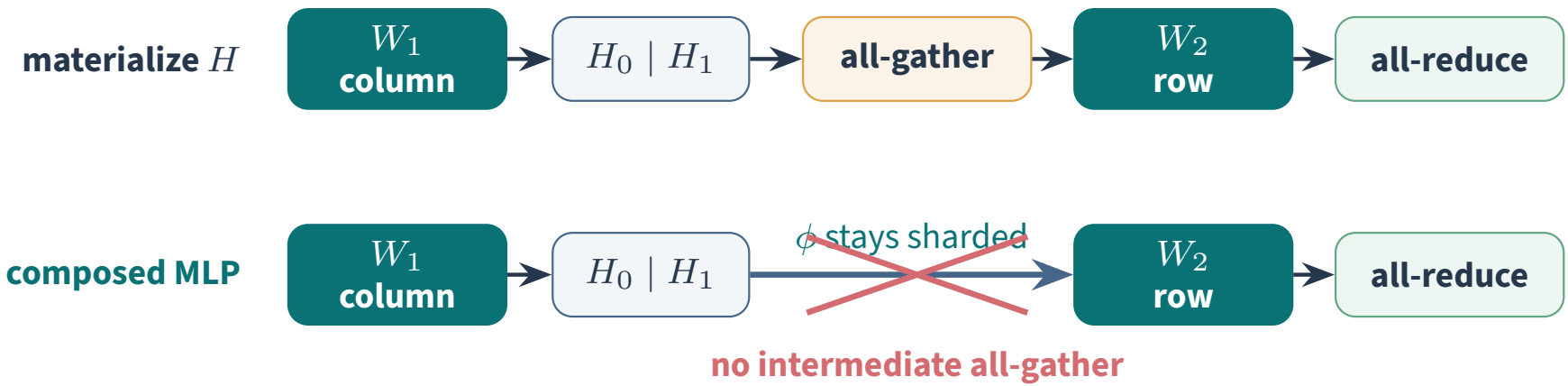
Follow a tensor-parallel MLP across two GPUs



Keep H_0, H_1 sharded between the two linear layers; sum the partial outputs before the residual add.

Forward pass; biases omitted.

Column then row parallelism forms an efficient sandwich



$H_i = \phi(XW_{1,i})$, then $Y = \sum_i H_i W_{2,i}$: two GEMMs, one forward collective.

Source: Hugging Face Ultra-Scale Playbook and Megatron-LM

PART 2

Inside a Transformer block

1. Start from one matrix multiplication
2. Inside a Transformer block
3. Cost and placement

Attention naturally shards by heads

- Column-shard the fused QKV projection so each rank owns a subset of heads.
- Compute attention locally for those heads.
- Row-shard the output projection and all-reduce its partial residual update.

Divisibility invariant

The number of query heads must divide by tensor-parallel degree. With GQA, key-value head partitioning imposes an additional constraint.

A layout ledger is the core debugging tool

Point in block	Layout	Local shape
residual input	replicated	$B \times T \times d$
QKV output	head/feature sharded	$B \times T \times 3d/p$
local attention output	head sharded	$B \times T \times d/p$
output projection partial	partial replicated shape	$B \times T \times d$
after all-reduce	replicated	$B \times T \times d$
MLP expansion	feature sharded	$B \times T \times 4d/p$

Every operation must declare what is sharded, what is replicated, and what collective changes that layout.

Sequence parallelism removes replicated token activations

LayerNorm, dropout, and some elementwise work do not require the full hidden tensor on every rank. Sequence parallelism shards the $B \times T$ token dimension between tensor-parallel regions.

Benefit

lower activation memory for operations that would otherwise remain replicated

Cost

reduce-scatter and all-gather transitions around tensor-parallel linear regions

Forward and backward collectives are conjugate

Forward operation	Forward layout change	Backward communication
identity with replicated input	replicated to sharded use	all-reduce input gradient
all-reduce sum	partial to replicated	identity on matching shard flow
reduce-scatter	replicated partials to shard	all-gather
all-gather	shards to replicated	reduce-scatter

Testing rule

Compare forward outputs, input gradients, and parameter gradients against an unsharded reference.

Checkpoint: layout trace

Think, then compare

Trace replicated or sharded status through a column-parallel MLP expansion, activation, row-parallel contraction, and residual add. At which point is communication unavoidable?

PART 3

Cost and placement

1. Start from one matrix multiplication
2. Inside a Transformer block
3. Cost and placement

Communication cost depends on activation size

A collective over $B \times T \times d$ elements moves bytes proportional to the activation tensor, not the parameter matrix.

$$S_{\text{act}} = B\,T\,d\,s$$

- Larger batch and context increase collective payload.
- Higher TP degree shrinks local matmuls and can reduce arithmetic efficiency.
- Collective latency and link bandwidth set the exposed cost.

Tensor parallelism belongs on the fastest links

Intra-node

High-bandwidth, low-latency GPU fabrics suit frequent activation collectives.

Across slow links

Small per-rank matmuls plus exposed all-reduces can erase the benefit quickly.

Keep TP inside a fast island when possible, then use data or pipeline parallelism across nodes.

Choose the smallest degree that solves the constraint

1. Determine whether a layer’s weights or activations fail to fit on one GPU.
2. Check head, hidden, and MLP divisibility for candidate degrees.
3. Map the group to the strongest available links.
4. Measure local matmul efficiency and collective exposure.
5. Verify numerics against the unsharded block.

Checkpoint: degree selection

Think, then compare

A model fits with TP=2, but TP=8 gives lower throughput. Explain the result using local matrix shapes, collective frequency, and topology.

Key takeaways

1. Column parallelism shards output features, while row parallelism sums partial outputs.
2. Their composition keeps intermediate MLP and attention shards local.
3. A layout ledger and unsharded numerical reference are essential correctness tools.
4. Tensor parallelism is a fine-grained method that needs large local work and fast links.